Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

**Звіт**
**До лабораторної роботи №5**
**Варіант 11**
**На тему**
**“Ліниві обчислення: генератори та itertools”**

Виконав:
Студентка групи ФЕП-21
Муковоз М. Т.

Перевірив:
Ас. Остріков Д. А.

Львів 2026

**Мета:** зрозуміти generator expressions, yield/yield from, ітератори та лінивість; уміти працювати з нескінченними послідовностями; освоїти базові будівельні блоки з itertools (islice, accumulate, count); реалізувати нескінченний потік факторіалів, sliding window, take/drop-практики.

## Теоретичні відомості

Ітератори дозволяють працювати з нескінченними джерелами даних завдяки лінивому обчисленню (значення генеруються по мірі запиту). Для створення лінивих послідовностей використовуються генератори (з ключовим словом yield) та готові інструменти з модуля itertools, такі як islice для підрізання потоку, accumulate для накопичення, count для нескінченного лічильника. Лінивість економить пам'ять та дозволяє створювати ефективні конвеєри обробки даних, де значення обчислюються виключно тоді, коли вони потрібні.

## Хід роботи

Згідно з варіантом 11 (Послідовність факторіалів), було реалізовано лінивий генератор factorial_stream, який будує нескінченний потік 0!, 1!, 2!... без повторного перерахунку проміжних значень. Для цього використано комбінацію нескінченного лічильника count та функції accumulate з operator.mul. Також реалізовано універсальні утиліти: take для відбору перших n елементів, drop для пропуску елементів та sliding_window на базі collections.deque для створення ковзних вікон без матеріалізації всього потоку.

streams.py > factorial_stream

```python
"""
Модуль для роботи з лінивими обчисленнями (генератори та itertools).
"""
from typing import Iterable, Iterator, TypeVar, Tuple
from itertools import islice, count, accumulate
from collections import deque
import operator

T = TypeVar("T")

def take(n: int, it: Iterable[T]) -> list[T]:
    """Взяти перші n елементів."""
    if n < 0:
        raise ValueError("n не може бути від'ємним")
    return list(islice(it, n))

```

```python

def drop(n: int, it: Iterable[T]) -> Iterator[T]:
    """Пропустити перші n елементів і повернути ітератор на решту."""
    if n < 0:
        raise ValueError("n не може бути від'ємним")
    it = iter(it)
    next(islice(it, n, n), None)
    return it

def sliding_window(it: Iterable[T], k: int) -> Iterator[Tuple[T, ...]]:
    """Повертає ковзне вікно розміру k."""
    if k <= 0:
        raise ValueError("Розмір вікна k має бути більше 0")
    it = iter(it)
    dq: deque[T] = deque(maxlen=k)
    for _ in range(k):
        try:
            dq.append(next(it))
        except StopIteration:
            return
    yield tuple(dq)
    for x in it:
        dq.append(x)
        yield tuple(dq)

def naturals(start: int = 1) -> Iterator[int]:
    """Нескінченний генератор натуральних чисел."""
    yield from count(start)

def factorial_stream() -> Iterator[int]:
    """
    Лінивий нескінченний потік факторіалів: 0!, 1!, 2!, 3!, ...
    Використовується count() та accumulate(operator.mul) для уникнення повторних обчислень.
    """
    yield 1
    yield from accumulate(naturals(1), operator.mul)
```

Для перевірки роботи створено файл main.py, у якому демонструється відбір перших 10 факторіалів, пропуск 5 значень і відбір наступних 5, а також побудова ковзних вікон розміром 3.

```python
main.py > main
from streams import factorial_stream, take, drop, sliding_window
from itertools import islice

def main():
    print("Лабораторна робота 5 (Варіант 11: Послідовність факторіалів) ===")

    stream = factorial_stream()
    first_10 = take(10, stream)
    print(f"Перші 10 факторіалів (0! - 9!): {first_10}")

    stream2 = factorial_stream()
    next_5 = take(5, drop(5, stream2))
    print(f"Факторіали з 5! по 9! (drop 5, take 5): {next_5}")

    stream3 = factorial_stream()
    windows = list(islice(sliding_window(stream3, 3), 5))
    print(f"Перші 5 ковзних вікон (k=3) для факторіалів:")
    for w in windows:
        print(f"  {w}")

if __name__ == "__main__":
    main()
```

```
Лабораторна робота 5 (Варіант 11: Послідовність факторіалів)
Перші 10 факторіалів (0! - 9!): [1, 1, 2, 6, 24, 120, 720, 5040, 40320, 362880]
Факторіали з 5! по 9! (drop 5, take 5): [120, 720, 5040, 40320, 362880]
Перші 5 ковзних вікон (k=3) для факторіалів:
  (1, 1, 2)
  (1, 2, 6)
  (2, 6, 24)
  (6, 24, 120)
  (24, 120, 720)
```

Усі реалізовані функції покрито автоматизованими тестами за допомогою pytest. Було написано спеціальний тест на перевірку "лінивості" обчислень (перевірка, що генератор обчислює рівно стільки елементів, скільки було запитано через take).

```
(.venv) C:\Users\MTV\Desktop\3 sem\FunkProg\Lab_05>pytest -q
........                                                                 [100%]

(.venv) C:\Users\MTV\Desktop\3 sem\FunkProg\Lab_05>
```

**Висновок:** У ході виконання лабораторної роботи №5 за варіантом 11 було успішно реалізовано концепції лінивих обчислень за допомогою генераторів та модуля itertools. Створено нескінченний потік факторіалів без повторного обчислення проміжних значень. Реалізовані функції take, drop та sliding_window дозволяють ефективно працювати з нескінченними колекціями, не перевантажуючи оперативну пам'ять. Автоматизовані тести підтвердили коректність результатів та довели ліниву природу створених потоків, що гарантує передбачуваність та оптимальне використання ресурсів.